Follow-Up Actions

Unitas System

Version 1.0

Prepared by Limbo Software Development

Rodrigo Alejandro Hurtado Cortés - A01713854

Alexis Yaocalli Berthou Haas - A01713458

Ana Camila Cuevas González - A01412609

Alejandro Contreras Magallanes - A01662743

April 28th, 2026

Table of Contents

1. Follow-Up Actions.....1
1.1 New functionalities..... 1
1.1.1 Employee's Expectations Report:..... 1
1.1.2 Connection of Highlight with Slack.....2
1.1.3 System Connection to Jira.....2
1.1.4 System Connection to GitHub..... 2
1.1.4 Report Edition Chatbot..... 3
1.2 Improvement of current functionalities..... 3
1.2.1 Connection of Activity Processor with Slack..... 3
1.2.2 Optimize search bar with indexing..... 4
2. Post-Delivery Support and Initial Maintenance..... 5
2.1 Scope of Support.....5
2.2 Response and Resolution Times..... 5
2.3 Considerations.....5

1.Follow-Up Actions

The present document outlines the set of proposed actions for the continued improvement and evolution of the **Unitas System**, developed for the organization Change.org. These proposals are based on internal development team analysis, feedback provided by Eng. Bernardo Laing, and additional "nice-to-have" functionalities identified during the final stages of the project.

It is important to note that all functionalities, ideas, and prototypes described herein are **not part of the original Software Requirements Specifications (SRS)**. All previously agreed-upon requirements between the development team and the client were fully implemented, tested, and approved prior to the final submission date of April 30th, 2026.

The following sections describe potential enhancements aimed at improving system capabilities, usability, and integration with external tools.

1.1 New functionalities

1.1.1 Employee's Expectations Report:

Development of an artificial intelligence–driven report that evaluates an employee's current performance against the expected responsibilities, competencies, and objectives associated with their or another role or organizational level.

This functionality would provide personalized insights and actionable recommendations, enabling employees to identify performance gaps and opportunities for growth.

Steps:

- Extend the current database schema to include role-based expectations and responsibilities for each employee type.
- Define and implement structured schemas (e.g., using Zod) to standardize AI input and output validation.
- Design and refine AI prompts for accurate comparison between expected and actual performance.
- Implement report processing logic within `aiWrapper.js` and the report module.
- Update the report generation components (EJS views and controllers) to support multiple report types dynamically.
- Validate AI outputs through testing and iterative prompt tuning.

1.1.2 Connection of Highlight with Slack

Implementation of an automated integration between Slack and the Unitas System to allow users to submit highlights directly from Slack channels.

This feature would streamline data entry and improve adoption by leveraging an existing communication platform.

Steps:

- Develop a Slack procedure with appropriate permissions (incoming webhooks or slash commands).
- Create a secure API endpoint in the Node.js backend to receive highlight submissions.
- Validate and sanitize incoming data to prevent malformed requests or injection attacks.
- Map Slack message structure to the internal highlight schema.
- Store submitted highlights in the MySQL database.
- Implement logging and error handling for traceability.
- Test the integration in a staging Slack workspace before deployment.

1.1.3 System Connection to Jira

Integration with Jira to enhance activity validation and improve the accuracy of AI-generated reports by incorporating real task data.

This feature would allow the system to extract relevant metadata from Jira links included in Daily Entries, Goals, Achievements or Highlights.

Steps:

- Research and configure access to the Jira REST API.
- Implement authentication (OAuth or API tokens) using environment variables.
- Develop backend services to fetch and parse Jira issue data.
- Extract relevant fields (status, assignee, progress, timestamps).
- Integrate extracted data into AI processing workflows.
- Handle invalid or inaccessible links gracefully.

1.1.4 System Connection to GitHub

Integration with GitHub to retrieve development activity data, providing additional context for performance analysis and reporting.

Steps:

- Configure GitHub API access using secure tokens stored in `.env`.

- Develop services to retrieve repository activity (commits, pull requests, issues).
- Associate GitHub activity with system users where applicable.
- Integrate retrieved data into report generation logic.
- Implement rate-limit handling and caching strategies.
- Validate and test data consistency.

1.1.4 Report Edition Chatbot

Development of an interactive chatbot within the report visualization interface, enabling users to modify and customize reports dynamically.

This feature would allow users to reorder sections, add new content, and persist customized report formats.

Steps:

- Design chatbot interaction flows and supported commands.
- Implement backend logic to process user instructions and modify report structure.
- Modify EJS views to reflect dynamic report updates.
- Ensure persistence of user-defined report formats.
- Conduct usability testing and refine interactions.

1.2 Improvement of current functionalities

1.2.1 Connection of Activity Processor with Slack

Enhancement of the existing Activity Processor to allow automatic submission of Daily Entries from Slack, replacing manual API testing via Postman.

This improvement would streamline data submission and increase usability.

Steps:

- Develop or modify the Slack workflow to trigger submissions.
- Connect Slack events to the existing backend endpoint.
- Ensure the JSON payload matches the current system schema.
- Validate incoming data on the backend.
- Implement logging and error handling.
- Conduct integration testing and user validation.

1.2.2 Optimize search bar with indexing

Enhancement of the current search functionality through the implementation of indexing mechanisms in the MySQL database to significantly improve query performance, reduce response time, and ensure scalability.

This improvement will enable faster data retrieval when users search across employees, projects, and activities, while reducing the need for full table scans.

Steps:

- Analyze current search queries in the Node.js backend to identify frequently used filters and fields.
- Identify key columns for indexing (e.g., employee names, project titles, activity descriptions, dates).
- Implement appropriate indexing strategies in MySQL:
    - Single-column indexes for frequently filtered fields
    - Composite indexes for multi-condition queries
    - FULLTEXT indexes for keyword-based searches
- Refactor backend queries to ensure proper index utilization (avoid functions on indexed columns).
- Optimize search logic to support efficient patterns (e.g., avoid `%keyword%` where possible).
- Use `EXPLAIN` to analyze query execution plans before and after indexing.
- Measure and compare performance improvements.
- Monitor database performance and adjust indexes to balance read and write efficiency.
- Ensure compatibility with existing API endpoints and frontend search components (EJS views).
- Document indexing strategy for future maintenance and scalability.

2.Post-Delivery Support and Initial Maintenance

As part of the Limbo Software Development team commitment to the quality and stability of the delivered solution, a **one (1) month post-delivery support period** is established, spanning from **May 1st to May 31st, 2026**. During this time, support will be provided for incident resolution, adjustments, and improvements to the implemented system.

The purpose of this period is to ensure proper system performance in a production environment, as well as to address potential improvement areas identified during initial use.

2.1 Scope of Support

During this period, the following activities are included:

- Resolution of errors in existing functionalities
- Minor frontend adjustments and user experience improvements
- Updates to existing components when necessary
- Analysis and implementation of enhancements or new features with controlled scope

2.2 Response and Resolution Times

Considering the part-time nature of the development team, the following service timeframes are defined:

- **Response Time:**
  Up to **48 business hours** from the moment a request is reported.
- **Minor Issues and Adjustments (Frontend / small fixes):**
  Estimated resolution time of **1 to 3 business days**.
- **New Features or Higher-Scope Changes:**
  Estimated development time of up to **7 business days**, depending on the complexity of the request.

2.3 Considerations

- Time estimates may vary depending on the complexity, technical dependencies, and validation requirements of each request.
- Requests may be prioritized based on impact and urgency.
- This support period does not include major system redesigns or large-scale feature development unless explicitly agreed upon.